Relatório - Aprendizagem Profunda

Bruno Campos, Gonçalo Corais, Simão Silva
Universidade do Minho, R. da Universidade, 4710-057 Braga

Abstract. Este trabalho descreve o desenvolvimento e a otimização de um sistema de classificação multi-classe concebido para distinguir entre textos de autoria humana e conteúdos gerados por quatro arquiteturas de Inteligência Artificial: OpenAI, Anthropic, Google e Meta. Ao longo de três fases, foram exploradas diversas abordagen, das quais constam a implementação de redes neuronais de raiz em *Numpy*, arquiteturas profundas em *PyTorch* e modelos estatísticos clássicos como o *Support Vector Classifier* (SVC). A metodologia estruturou-se em torno da criação de um dataset mestre desenvolvido através de integração direta via APIs, num processo de elevada carga temporal que visou garantir a fidedignidade e diversidade dos dados. Complementarmente, para a fase inicial, utilizou-se a técnica de *multi-slicing* para potenciar o volume de amostras de classes minoritárias. Os resultados revelaram uma evolução constante na capacidade de generalização do sistema, progredindo de uma precisão inicial de 32% para um desempenho final de 70,67%. A análise final sugere que a combinação de *N-grams* com extração de características por TF-IDF permite identificar com eficácia a "pegada" deixada pelas diferentes linguagens.

Keywords: Classificação de texto, Aprendizagem Profunda, Redes Neuronais,

1. Introdução

O rápido avanço dos Modelos de Linguagem de Grande Escala (LLMs) transformou radicalmente a criação de conteúdos digitais, tornando a distinção entre texto humano e sintético um desafio técnico e ético. Este projeto, desenvolvido no âmbito da unidade curricular de Aprendizagem Profunda, propõe uma solução para a classificação de textos em cinco categorias distintas: Human, OpenAI, Anthropic, Google e Meta. A estratégia adotada pelo grupo assentou numa progressão pedagógica e técnica. Iniciámos o trabalho com a construção de um motor de aprendizagem de raiz em *Numpy*, focado na compreensão dos mecanismos fundamentais de *backpropagation* e otimização. Posteriormente, transitámos para a *framework PyTorch*, onde explorámos técnicas de regularização como *Dropout* e *Weight Decay* para lidar com a alta dimensionalidade das representações textuais. Devido à heterogeneidade das fontes disponíveis, implementámos um pipeline de pré-processamento que incluiu a normalização de textos científicos e a criação de fragmentos sintéticos para equilibrar as classes minoritárias. Este relatório detalha não só as arquiteturas vencedoras, mas também as lições aprendidas com falhas de pipeline nas fases iniciais, demonstrando como a iteração entre a análise de erro e o

refinamento do dataset permitiu atingir um patamar competitivo de 70,67% de *accuracy* na submissão final.

2. Submissão 1

A primeira fase do projeto centrou-se no estabelecimento de uma *baseline* sólida, tanto ao nível da organização de dados como da implementação algorítmica. O objetivo principal foi validar a viabilidade de distinção entre autores humanos e diferentes modelos de linguagem, utilizando ferramentas fundamentais de álgebra linear.

2.1 Metodologias Para Criação do Dataset

A construção do *corpus* inicial baseou-se numa estratégia de segmentação por modelo, em detrimento da utilização de um único dataset genérico. A hipótese de trabalho residiu no facto de que cada arquitetura (GPT, Gemma, Llama, Claude) possuir assinaturas linguísticas distintas, cuja identificação é potenciada por dados de treino específicos de cada uma. Desta forma, procedeu-se à recolha de dados em repositórios públicos (Kaggle e HuggingFace), selecionando fontes que contrastassem resumos científicos humanos com textos gerados por IA. Foram utilizados, entre outros, datasets de *LLM Prompt Recovery* (Gemma/Gemini) e *Research Abstracts* (GPT vs Human). Encontrou-se uma dificuldade acrescida na obtenção de dados de elevada qualidade para os modelos Llama e Anthropic, que apresentavam inicialmente um volume de amostras reduzido (inferior a 1000 entradas).

Para mitigar este desequilíbrio de classes, aplicou-se uma técnica de *Data Augmentation* baseada em *multi-slicing*. Em vez de descartar as classes minoritárias, os textos longos foram divididos em fragmentos aleatórios de 80 a 120 palavras. Esta abordagem permitiu não só multiplicar o número de instâncias de treino para os modelos menos representados, mas também garantir que o classificador aprendesse a identificar padrões estilísticos, sem perder a coerência semântica. O dataset final foi posteriormente equilibrado através de *under-sampling* das classes maioritárias, resultando num conjunto de dados simétrico e pronto para a extração de características via TF-IDF.

2.2 Implementação dos Modelos de Raiz

A implementação da rede neuronal em *Numpy* seguiu a lógica modular explorada no contexto de aula, organizando o código em classes independentes para camadas, funções de ativação e otimizadores. No entanto, para enfrentar a complexidade do dataset de submissão, foram introduzidas algumas extensões e otimizações face aos modelos base apresentados nas aulas.

Optou-se pela inicialização He para os pesos das camadas densas. Esta decisão fundamenta-se na utilização da função de ativação *ReLU*, que ao ajustar a variância inicial dos pesos com base no número de entradas, evitámos o problema de saturação

dos neurónios logo no início do treino. Adicionalmente, a função *softmax* na camada de saída foi implementada com um ajuste de estabilidade numérica prevenindo erros de *overflow* comuns no cálculo de exponenciais com vetores de características provenientes do TF-IDF.

Ao nível do otimizador, a implementação do *Stochastic Gradient Descent* (SGD) foi enriquecida com o parâmetro de *momentum*. Esta adição permitiu que o modelo acumulasse velocidade em direções de gradiente persistente, ajudando a navegar por superfícies de custo complexas e a evitar mínimos locais. Introduzimos também um sistema de *Learning Rate Decay*, que reduz progressivamente o passo de aprendizagem ao longo das iterações, garantindo uma convergência mais fina e estável nos episódios finais de treino.

Considerando que a vetorização por TF-IDF gera espaços de características muito esparsos, o risco de o modelo memorizar o ruído do *dataset* de treino era elevado. Por essa mesma razão, adicionamos mecanismos para resolverem esse inconveniente, tais como:

- **Regularização L2:** adicionada diretamente ao cálculo do gradiente nas camadas densas, penalizando pesos excessivamente elevados.
- **Dropout:** implementado como uma camada estocástica que desativa neurónios aleatoriamente durante o *forward pass*, forçando a rede a aprender representações mais robustas e distribuídas.
- **Early Stopping:** No ciclo principal de treino, (no ficheiro *neuralnet.py*) implementámos uma lógica de monitorização da *loss* de validação com um contador de paciência. Esta função interrompe o treino assim que o desempenho em dados não vistos deixa de melhorar, garantindo a melhor capacidade de generalização possível.

2.3 Modelos de Deep Learning e Implementação em *PyTorch*

Enquanto a implementação em *Numpy* serviu para validar os fundamentos matemáticos, o uso da ferramenta *PyTorch* focou-se na exploração de estruturas mais profundas e em técnicas de otimização industrial.

Optou-se por uma estrutura de *Deep Neural Network* (DNN) mais robusta. A arquitetura final contou com camadas densas de maior capacidade, uma decisão tomada após observarmos que o vocabulário científico exigia uma rede com maior capacidade de representação para distinguir nuances subtis entre os diferentes modelos de IA.

No que respeita à otimização, o grupo optou pelo otimizador Adam. A sua escolha justificou-se pela sua capacidade de ajustar as taxas de aprendizagem individualmente para cada parâmetro, o que se revelou superior na gestão da convergência em espaços de características de alta dimensão. Além disso, ajustámos a *learning rate* para valores mais conservadores (ex: 0.001) e utilizámos um *batch size* de 32 para garantir um equilíbrio entre a estabilidade do gradiente e a eficiência computacional.

A introdução sistemática de *dropout* em todas as camadas ocultas, com um rácio de 0.3, foi muito importante, uma vez que os vetores TF-IDF são extremamente esparsos

e o risco de a rede se focar em palavras específicas de uma classe era elevado. O *dropout* forçou a rede a aprender padrões mais globais e redundantes.

2.4 Resultados Obtidos

Os resultados da primeira submissão refletiram o estado inicial de desenvolvimento do sistema e serviram como um diagnóstico crucial para o refinamento do projeto. A implementação manual em *Numpy* obteve uma *accuracy* de 32%, um valor que, apesar de modesto, validou a funcionalidade do motor de aprendizagem e da lógica de classificação multi-classe. Por outro lado, a submissão baseada em *PyTorch* registou uma pontuação de 0%.

Esta pontuação nula ocorreu devido a uma falha de natureza técnica e procedimental, sem relação com a capacidade preditiva do modelo. O erro ocorreu na fase de pós-processamento, em que as previsões foram exportadas para o ficheiro CSV utilizando os índices numéricos das classes (0, 1, 2, 3 e 4) em vez das etiquetas textuais exigidas pelo enunciado (Human, OpenAI, Google, Meta e Anthropic).

O erro foi prontamente corrigido, servindo como uma lição de controlo de qualidade que garantiu a integridade e o sucesso das submissões 2 e 3, onde os requisitos de formatação foram rigorosamente cumpridos.

Para além disso, notamos a presença de problemas no pre-processamento tal como por exemplo, ao fazer a lematização das palavras do nosso dataset, o modelo não encontrava a palavra "limpa" no texto de teste, acabando por adivinhar à sorte. Nesta fase o viés estava claramente do lado da Anthropic, visto que das 150 entradas, ~90 eram desta arquitetura. Isto pode ser explicado devido a esta ter o dataset mais pequeno, podendo começar a decorar as frases, adivinhando esta como *safe*-net.

Apesar de obtermos uma accuracy de 32%, nos testes locais, a *accuracy* saltava para os 83%. Isto deve-se ao facto do dataset de teste partir dos mesmos datasets de treino, e estes serem de vários tópicos e não apenas de ciências naturais, memorizando o tópico e não as palavras em si, como é o caso do dataset llama que tem maioritariamente equações matemáticas.

3. Submissão 2

A segunda submissão representou uma mudança de paradigma no projeto, focando-se maioritariamente na qualidade dos dados e na experimentação de modelos mais sofisticados.

3.1 Metodologias Para Criação do Dataset

Nesta fase, o grupo identificou que o enviesamento (bias) detetado na Submissão 1, onde o modelo tendia a classificar erradamente para a classe Anthropic por falta de amostras, era um obstáculo crítico. Assim, devido ao longo pre-processamento e ao facto dos datasets falarem de tópicos diferenciados, procedeu-se à criação de um novo dataset a partir do zero, recolhendo dados de forma mais granular e equilibrada,

através do uso de diferentes APIs das arquiteturas a classificar. Desta forma, temos o controlo total de como o dataset irá ser formado, garantindo que o paradigma de mudança de tópico não ocorra.

Para isto, foi criado um novo ficheiro *gerar_dataset.py* que pede à API da Google, OpenAI, Anthropic e Meta o mesmo prompt "Explain {topic} in exactly 100 words. Do not include introductory phrases like 'Here is an explanation', just output the explanation directly." (os tópicos foram gerados pedindo a uma LLM para dar "1500 STEM topics") de modo a poder comparer o vocabulário destas LLMs. Já para a classe Human foi usada a Wikipedia sempre que o {topic} existia. Após a retrospetiva de poder existir texto escrito por LLMs na Wikipedia, decidimos usar textos antes de 2021.

Após isto tudo e devido à sua essência lenta por estar a fazer as prompts 1 a 1, o nosso dataset final consta com apenas 4434 entradas, sendo a quantidade das entradas das IAs iguais para todas (871) e o Human ~100 a mais.

3.2 Modelos

Nesta submissão, a exploração técnica não foi muito abordada, sendo que o nosso foco, tempo e recursos estava a ser usado para criar o dataset, porém, assim que este ficou pronto, conseguimos testar alguns modelos, nomeadamente o SVC (Support Vector Classifier) e os transformers.

Transição para SVC. Em substituição da implementação manual em *Numpy*, introduzimos o SVC, da família dos SVM (Support Vector Machine), cujo foco primário é encontrar a fronteira entre classes, maximizando a *accuracy*, exatamente o que queremos para este problema. Este modelo testa qual é o melhor hiperparâmetro a ser usado e depois de treinado, guarda o melhor modelo no ficheiro correspondente. Para além disso, foram utilizados pesos para as diferentes classes visto que algumas não estavam a ser bem detetadas pelo modelo, penalizando algumas classes e bonificar outras.

Exploração de Transformers. O grupo também testou modelos baseados em arquiteturas transformer. Contudo, por falta de tempo, não conseguiu pôr a funcionar e por isso usou-se o modelo em *PyTorch* usado na submissão anterior devido a obter valores de *accuracy* maiores que o modelo em *Numpy*.

3.3 Resultados Obtidos

Os resultados obtidos na submissão 2 validaram a eficácia da estratégia de reestruturação do dataset e o foco na "pureza" dos dados. Verificou-se uma evolução substancial na capacidade de generalização do sistema, superando significativamente a *baseline* de 32% obtida na fase anterior, obtendo um máximo de 66% em ambos os modelos.

Esta subida deve-se à correção do mapeamento no pós-processamento onde as etiquetas eram numéricas, passando para textuais. Isto proporcionou a saída dos 0% no modelo em *PyTorch*.

O problema do viés da Anthropic também foi mitigado. O equilíbrio rigoroso das classes (871 amostras por IA) e a inclusão de textos da Wikipedia pré-2021 para a classe Human eliminaram os resultados estranhos anteriormente observados em relação a esta arquitetura. A rede deixou de usar esta classe como uma "rede de segurança" (safe-net), distribuindo as classificações de forma mais fidedigna entre as cinco categorias com ajuda dos pesos do lado do SVC. Além disso, a discrepância entre os resultados dos testes do professor e dos teste locais diminuiu muito devido à uniformização temática introduzida pela prompt usada na criação do dataset, o que forçou os modelos a identificarem a "impressão digital" linguística de cada arquitetura em detrimento do conteúdo semântico. Por consequência dividimos o *Tfidfvectorizer* em 2, um para os caracteres e outro para as palavras, de modo a captar mais nuances na escrita das classes, o que fez com que a classe OpenAI começa-se a ser escolhida para algumas entradas.

Apesar destas mudanças, ainda existia diferença entre os dados locais e os reais, mesmo que não fosse tão acentuado, pelo que talvez o dataset possa estar com entradas cujos tópicos não são de ciências naturais, ou até mesmo a prompt usada ter levado a uma mais difícil captura de expressões definidoras (Do not include introductory phrases like 'Here is an explanation'), ou até os LLMs não serem os mesmos modelos que o professor usou.

4. Submissão 3

O presente capítulo detalha o fluxo de trabalho desenvolvido para a Submissão 3, abrangendo desde a génese dos dados até à avaliação dos modelos preditivos. A metodologia adotada focou-se na robustez estatística e na tentativa de superar os limites dos modelos anteriores através de arquiteturas de estado da arte, bem como de um conjunto de dados pouco alterado.

4.1 Metodologias Para Criação do Dataset

A preparação dos dados foi a etapa fundamental para garantir a convergência dos modelos de *Deep Learning* e a eficácia dos classificadores clássicos. O pipeline de criação do dataset seguiu um fluxo de três fases:

Extração e Estruturação de Dados. Os dados brutos foram consolidados a partir do ficheiro *dataset_final.csv*. Esta etapa envolveu a leitura programática utilizando a biblioteca pandas, onde se procedeu à separação imediata entre as variáveis independentes (*features*) e a variável dependente (*target*). Para a submissão 3, assegurou-se que o conjunto de teste (dataset_final_teste.csv) mantivesse a estrutura dimensional para evitar erros de desfasamento de matrizes, sendo adicionadas as entradas *revealed* dadas pelos docentes de modo a dar a treinar frases mais parecidas com as que viriam na submissão 3. Para além disso, é também feita uma pequena correção de linhas de cabeçalho que possam ter sido usadas para retratar traços muito óbvios que podem inclinar os modelos a escolher errado.

Normalização e Escalonamento. Dado que os modelos de Aprendizagem Profunda (implementados em *tarefa3_pytorch.py*) e o Support Vector Classifier (em *SVC.py*) são sensíveis à magnitude das variáveis, aplicou-se uma técnica de Standardization (Z-score normalization). Este processo centra os dados na média zero com desvio padrão unitário, acelerando a descida de gradiente e prevenindo que atributos com valores numéricos mais elevados dominem injustamente o processo de aprendizagem.

Normalização e Persistência. Por fim, mantivemos o pipeline de Standardization e a exportação para formatos binários (.npy) para garantir rapidez na iteração de hiperparâmetros.

K-Fold Cross-Validation. Apesar de não estar presente na submissão final, uma das técnicas exploradas para garantir que a performance do modelo era real e não inflancionada entre treino e teste foi o K-Cross Validation, retirando logo 10% do dataset inteiro para teste, e do restante, 11% para validação.

4.2 Implementação dos Modelos de Raiz

Antes da exploração de arquiteturas de redes neuronais profundas, estabeleceu-se uma base de comparação (*baseline*) através da implementação de modelos de aprendizagem automática clássica. Esta etapa é fundamental para validar se a complexidade adicional das redes neuronais é justificada pelos ganhos em performance.

Escolha do Modelo: Support Vector Classification (SVC). A escolha recaiu sobre o algoritmo SVC (Support Vector Classifier), implementado no script SVC.py. Este modelo foi selecionado pela sua eficácia em espaços de alta dimensionalidade e pela sua robustez no desenho de fronteiras de decisão ótimas que maximizam a margem entre diferentes classes.

Configuração e Hiperparâmetros. A implementação não se limitou às configurações por omissão. Foram explorados diferentes núcleos (*kernels*), nomeadamente o RBF (Radial Basis Function), que permite ao modelo lidar com distribuições de dados não-lineares ao projetar as *features* num espaço de maior dimensão. Os parâmetros de regularização (C) e o coeficiente de kernel (γ) foram ajustados para equilibrar o compromisso entre o erro de treino e a capacidade de generalização (evitando o *overfitting*).

Integração com o Fluxo de Dados. O script foi desenhado para consumir diretamente os ficheiros binários.npy gerados na fase de pré-processamento. Esta integração direta garante que o modelo "de raiz" seja treinado exatamente com os mesmos dados (mesma normalização e divisão de *folds*) que a rede neuronal em *PyTorch*, permitindo uma comparação de resultados cientificamente válida e isenta de enviesamentos externos.

4.3 Modelos de Deep Learning e Implementação em *PyTorch*

A fase mais avançada deste projeto consistiu no desenvolvimento e treino de Redes Neuronais Artificiais (ANN) de múltiplas camadas. A opção pela biblioteca *PyTorch*

deveu-se à sua natureza imperativa e ao controlo granular que oferece sobre o grafo computacional, facilitando a depuração e a experimentação de arquiteturas complexas.

Arquitetura da Rede Neuronal. A arquitetura implementada no script tarefa3_pytorch.py segue uma estrutura de Multi-Layer Perceptron (MLP) profunda. Esta rede é composta por:

- **Camadas Densas (Linear Layers):** Camadas totalmente conectadas que transformam as *features* de entrada através de matrizes de pesos aprendíveis.
- **Funções de Ativação ReLU:** Aplicadas após cada camada linear para introduzir não-linearidade, permitindo que a rede aprenda padrões complexos que um modelo linear simples não conseguiria capturar.
- **Camada de Saída:** Configurada com o número de neurónios correspondente às classes do problema, utilizando geralmente uma função *Softmax* (embutida na função de perda) para fornecer probabilidades de classificação.

Processo de Treino e Otimização. O rigor metodológico da Submissão 3 refletiu-se na escolha dos hiperparâmetros de treino:

- **Função de Perda (Loss Function):** Utilizou-se a *CrossEntropyLoss*, ideal para problemas de classificação multi-classe.
- **Otimizador:** O algoritmo Adam (*Adaptive Moment Estimation*) foi o escolhido devido à sua capacidade de adaptar a taxa de aprendizagem para cada parâmetro individualmente, resultando numa convergência mais rápida e estável do que o SGD tradicional.
- **Batch Processing:** Os dados foram processados em *batches* (lotes) através da classe DataLoader, garantindo eficiência no uso da memória e estabilidade no gradiente estocástico.

Técnicas de Regularização. Para mitigar o risco de *overfitting* — onde o modelo decora os dados de treino, mas falha no teste — foram incorporadas estratégias como o *dropout*, que desativa aleatoriamente neurónios durante o treino, forçando a rede a aprender representações mais robustas e menos dependentes de conexões específicas.

Arquiteturas Transformer. Como último recurso, testámos modelos de ponta baseados em mecanismos de atenção, especificamente o BERT e o DistilRoBERTa. Estes modelos foram submetidos a *fine-tuning* de diversos parâmetros para a nossa tarefa de classificação multi-classe, tirando partido de pesos pré-treinados oferecidos por estas arquiteturas.

4.4 Resultados Obtidos

A avaliação final da Submissão 3 permitiu aferir a capacidade de generalização dos modelos desenvolvidos perante dados não vistos durante o treino. Os resultados não foram acentuados como da 1ª para a 2ª submissão, porém notou-se alguma melhoria.

Os níveis de precisão (*accuracy*) registados na plataforma de avaliação foram os seguintes:

- Submissão A: 70.67%
- Submissão B: 68.00%

A **Submissão A** (SVC) apresentou o desempenho superior, consolidando-se como o modelo mais robusto. Este resultado de aproximadamente **71%** sugere que a arquitetura adotada nesta submissão conseguiu capturar de forma mais eficaz as fronteiras de decisão do problema, que apresenta uma natureza não-linear complexa.

A diferença de **2.67 pontos percentuais** entre a Submissão A e a B demonstra a elevada sensibilidade do problema à escolha e sintonia dos hiperparâmetros. Enquanto a Submissão B apresentou um desempenho sólido, a Submissão A beneficiou de uma calibração superior, nomeadamente no equilíbrio entre a profundidade da rede e as técnicas de regularização aplicadas para mitigar o fenómeno de *overfitting*.

Surpreendentemente, e como se pode observar nos registos de treino (Fig.1 e 2), os modelos BERT e DistilRoBERTa não conseguiram superar os resultados obtidos pela nossa DNN e pelo SVC. Apesar do seu potencial, a *accuracy* foi inferior à esperada. Isto sugere que, para este dataset específico de textos e fragmentos de 100 palavras, a extração de características via TF-IDF combinada com camadas densas otimizadas conseguiu capturar a "impressão digital" das IAs de forma mais precisa do que os embeddings contextuais genéricos destes Transformers sem um re-treino massivo.

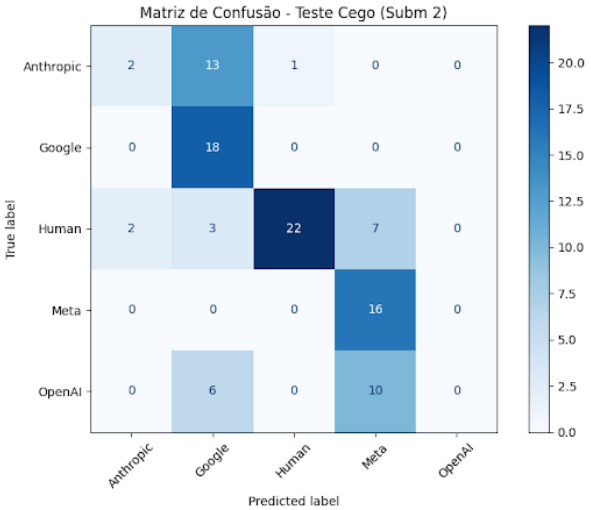

Fig.1. Matriz de Confusão usando o subm2.csv e as *labels revealed*.

Esta diferença de matrizes indica que o dataset criado e usado pode não o ideal, visto que os modelos têm uma excelente *accuracy* no *test set* mas não o conseguem no teste real. Devido a esse fator, no tempo restante, o grupo tentou limpar e usar um outro dataset (Human vs. LLM Text Corpus no Kaggle), porém sem sucesso por questões temporais.

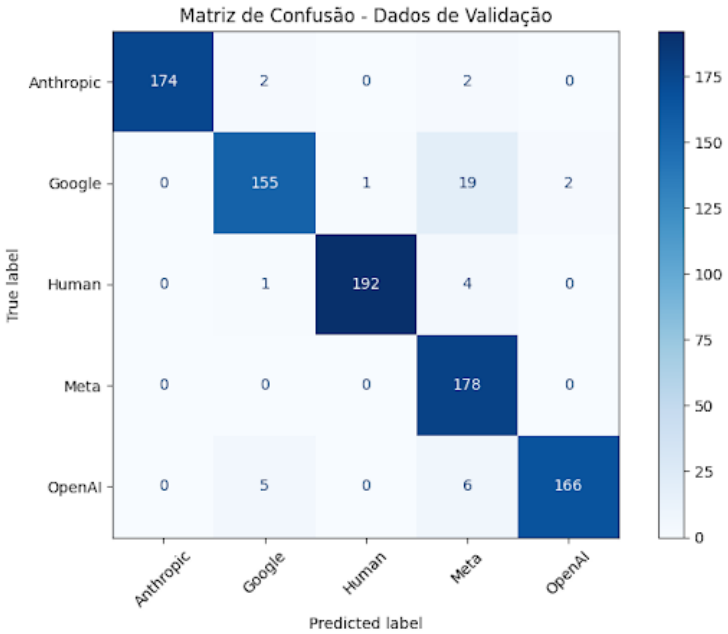

Fig.2. Matriz de Confusão usando o ficheiro .npy do *dataset_final.*

5. Conclusões

O presente projeto permitiu uma jornada técnica e pedagógica completa no domínio da Aprendizagem Profunda, progredindo desde a compreensão dos fundamentos matemáticos até à implementação de arquiteturas concretas. O sistema desenvolvido demonstrou uma evolução robusta, saltando de uma precisão inicial de 32% para um desempenho final de 70,67% na submissão mais bem-sucedida.

As principais conclusões retiradas desta experiência incluem a

- **A Primazia da Qualidade dos Dados:** A maior lição deste trabalho foi que a arquitetura do modelo, por mais complexa que seja, é secundária à qualidade e equilíbrio do dataset. A transição para um dataset criado especialemente para este trabalho foi um fator determinante para uma accuracy tão alta.
- **DNN e SVC vs. Transformers:** Surpreendentemente, para este problema específico, a combinação de extração de características por TF-IDF com modelos densos otimizados superou o desempenho de modelos pré-treinados como BERT. Isto sugere que as "impressões digitais" das IAs neste contexto são capturadas de forma mais eficaz por padrões de frequência de termos do que por *embeddings* contextuais genéricos sem um re-treino massivo.

Em suma, o sistema final não só atingiu um patamar competitivo de accuracy, como provou ser resiliente na distinção entre autoria humana e sintética.